Preprocessing: import modules, read the image, and create the output directory for pictures.

```
import cv2
import numpy as np
import os
import math

path = r"C:\ntu_csie\1141\cv\lena.bmp"
img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
h, w = img.shape

output_dir = r"C:\ntu_csie\1141\cv\output\hw9"
os.makedirs(output_dir, exist_ok=True)
```

Use the kernels and formulas defined in CV1_CH7_2025.pptx to do convolution on lena.bmp and do corresponding processing on it.

```
# (a) Robert's Operator: 12
a = np.full((h, w), 255, dtype=np.uint8)
r1 = np.array([
    [-1, 0],
    [0, 1]
])
r2 = np.array([
    [0, -1],
    [1, 0]
])
threshold = 12
for i in range(h-1):
    for j in range(w-1):
        region = img[i:i+2, j:j+2]
        x = np.sum(r1*region)
        y = np.sum(r2*region)
        gradient = np.sqrt(x**2 + y**2)
        if gradient >= threshold:
            a[i, j] = 0
savepath = os.path.join(output_dir, "a.bmp")
cv2.imwrite(savepath, a)
```

```
# (b) Prewitt's Edge Detector: 24
b = np.full((h, w), 255, dtype=np.uint8)
p1 = np.array([
    [-1, -1, -1],
    [0, 0, 0],
    [1, 1, 1],
])
p2 = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1],
])
threshold = 24
for i in range(1,h-1):
    for j in range(1,w-1):
        region = img[i-1:i+2, j-1:j+2]
        x = np.sum(p1*region)
        y = np.sum(p2*region)
        gradient = np.sqrt(x**2 + y**2)
        if gradient >= threshold:
            b[i, j] = 0
savepath = os.path.join(output_dir, "b.bmp")
cv2.imwrite(savepath, b)
```

```python
# (c) Sobel's Edge Detector: 38
c = np.full((h, w), 255, dtype=np.uint8)
p1 = np.array([
    [-1, -2, -1],
    [0, 0, 0],
    [1, 2, 1],
])
p2 = np.array([
    [-1, 0, 1],
    [-2, 0, 2],
    [-1, 0, 1],
])
threshold = 38
for i in range(1,h-1):
    for j in range(1,w-1):
        region = img[i-1:i+2, j-1:j+2]
        x = np.sum(p1*region)
        y = np.sum(p2*region)
        gradient = np.sqrt(x**2 + y**2)
        if gradient >= threshold:
            c[i, j] = 0
savepath = os.path.join(output_dir, "c.bmp")
cv2.imwrite(savepath, c)
```

```python
# (d) Frei and Chen's Gradient Operator: 30
d = np.full((h, w), 255, dtype=np.uint8)
sqrt2 = math.sqrt(2)
f1 = np.array([
    [-1, -sqrt2, -1],
    [0, 0, 0],
    [1, sqrt2, 1],
])
f2 = np.array([
    [-1, 0, 1],
    [-sqrt2, 0, sqrt2],
    [-1, 0, 1],
])
threshold = 30
for i in range(1,h-1):
    for j in range(1,w-1):
        region = img[i-1:i+2, j-1:j+2]
        x = np.sum(f1*region)
        y = np.sum(f2*region)
        gradient = np.sqrt(x**2 + y**2)
        if gradient >= threshold:
            d[i, j] = 0
savepath = os.path.join(output_dir, "d.bmp")
cv2.imwrite(savepath, d)
```

```python
# (e) Kirsch's Compass Operator: 135
e = np.full((h, w), 255, dtype=np.uint8)
k0 = np.array([
    [-3,-3,5],
    [-3,0,5],
    [-3,-3,5],
])
k1 = np.array([
    [-3,5,5],
    [-3,0,5],
    [-3,-3,-3],
])
k2 = np.array([
    [5,5,5],
    [-3,0,-3],
    [-3,-3,-3],
])
k3 = np.array([
    [5,5,-3],
    [5,0,-3],
    [-3,-3,-3],
])
k4 = np.array([
    [5,-3,-3],
    [5,0,-3],
    [5,-3,-3],
])
k5 = np.array([
    [-3,-3,-3],
    [5,0,-3],
    [5,5,-3],
])
k6 = np.array([
    [-3,-3,-3],
    [-3,0,-3],
    [5,5,5],
])
k7 = np.array([
    [-3,-3,-3],
    [-3,0,5],
    [-3,5,5],
])

threshold = 135
for i in range(1,h-1):
    for j in range(1,w-1):
        region = img[i-1:i+2, j-1:j+2]
        g = []
        mx = float('-inf')
        for k in [k0, k1, k2, k3, k4, k5, k6, k7]:
            g = np.sum(k*region)
            mx = max(mx, g)
        if mx >= threshold:
            e[i, j] = 0
savepath = os.path.join(output_dir, "e.bmp")
cv2.imwrite(savepath, e)
```

```python
# (f) Robinson's Compass Operator: 43
f = np.full((h, w), 255, dtype=np.uint8)
r0 = np.array([
    [-3,-3,5],
    [-3,0,5],
    [-3,-3,5],
])
r1 = np.array([
    [-3,5,5],
    [-3,0,5],
    [-3,-3,-3],
])
r2 = np.array([
    [5,5,5],
    [-3,0,-3],
    [-3,-3,-3],
])
r3 = np.array([
    [5,5,-3],
    [5,0,-3],
    [-3,-3,-3],
])
r4 = -r0
r5 = -r1
r6 = -r2
r7 = -r3

threshold = 43
for i in range(1,h-1):
    for j in range(1,w-1):
        region = img[i-1:i+2, j-1:j+2]
        g = []
        mx = float('-inf')
        for r in [r0, r1, r2, r3, r4, r5, r6, r7]:
            g = np.sum(r*region)
            mx = max(mx, g)
        if mx >= threshold:
            f[i, j] = 0
savepath = os.path.join(output_dir, "f.bmp")
cv2.imwrite(savepath, f)
```

```python
# (g) Nevatia-Babu 5x5 Operator: 12500
g = np.full((h, w), 255, dtype=np.uint8)
nb0 = np.array([
    [100,100,100,100,100],
    [100,100,100,100,100],
    [0,0,0,0,0],
    [-100,-100,-100,-100,-100],
    [-100,-100,-100,-100,-100],
])
nb30 = np.array([
    [100,100,100,100,100],
    [100,100,100,78,-32],
    [100,92,0,-92,-100],
    [32,-78,-100,-100,-100],
    [-100,-100,-100,-100,-100],
])
nb60 = np.array([
    [100,100,100,32,-100],
    [100,100,92,-78,-100],
    [100,100,0,-100,-100],
    [100,78,-92,-100,-100],
    [100,-32,-100,-100,-100],
])
nb_90 = np.array([
    [-100,-100,0,100,100],
    [-100,-100,0,100,100],
    [-100,-100,0,100,100],
    [-100,-100,0,100,100],
    [-100,-100,0,100,100],
])
nb_60 = np.array([
    [-100,32,100,100,100],
    [-100,-78,92,100,100],
    [-100,-100,0,100,100],
    [-100,-100,-92,78,100],
    [-100,-100,-100,-32,100],
])
nb_30 = np.array([
    [100,100,100,100,100],
    [-32,78,100,100,100],
    [-100,-92,0,92,100],
    [-100,-100,-100,-78,32],
    [-100,-100,-100,-100,-100],
])

threshold = 12500
for i in range(2,h-2):
    for j in range(2,w-2):
        region = img[i-2:i+3, j-2:j+3]
        mx = float('-inf')
        for nb in [nb0,nb30,nb60,nb_90,nb_60,nb_30]:
            value = np.sum(nb*region)
            mx = max(mx, value)
        if mx >= threshold:
            g[i, j] = 0
savepath = os.path.join(output_dir, "g.bmp")
cv2.imwrite(savepath, g)
```